

Student ID: 2408861 Student Name: Prachi Rajbanshi

```
pip install keras tensorflow
```

[Show hidden output](#)

```
import os
```

```
train_dir = "/content/drive/MyDrive/AIandML/FruitinAmazon/train"
classes = os.listdir(train_dir)
print(classes)
```

```
['guarana', 'tucuma', 'pupunha', 'acai', 'cupuacu', 'graviola']
```

```
import os
```

```
test_dir = "/content/drive/MyDrive/AIandML/FruitinAmazon/test"
classes = os.listdir(test_dir)
print(classes)
```

```
['tucuma', 'acai', 'guarana', 'graviola', 'pupunha', 'cupuacu']
```

```
import matplotlib.pyplot as plt
import random
from PIL import Image
```

```
plt.figure(figsize=(10,5))
```

```
for i, cls in enumerate(classes):
    img_path = os.path.join(train_dir, cls, random.choice(os.listdir(os.path.join(train_dir, cls))))
    img = Image.open(img_path)
    plt.subplot(2,3,i+1)
    plt.imshow(img)
    plt.title(cls)
    plt.axis("off")
```

```
plt.show()
```



```
from PIL import Image
import os
```

```
corrupted = []
```

```
for cls in classes:
    folder = os.path.join(train_dir, cls)
    for img_name in os.listdir(folder):
        img_path = os.path.join(folder, img_name)
        try:
            img = Image.open(img_path)
            img.verify()
```

```

    except:
        corrupted.append(img_path)
        os.remove(img_path)
        print(f"Removed corrupted image: {img_path}")

if not corrupted:
    print("No corrupted images found.")

```

No corrupted images found.

```
import tensorflow as tf
```

```
# Define parameters
```

```

img_height = 128
img_width = 128
batch_size = 32
validation_split = 0.2

```

```

# Load training
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    validation_split=validation_split,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size
)

```

Found 90 files belonging to 6 classes.
Using 72 files for training.

```

# Load validation dataset (Split from training dataset)
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    validation_split=validation_split,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size
)

```

Found 90 files belonging to 6 classes.
Using 18 files for validation.

```

# Load test data
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    shuffle=False # IMPORTANT: don't shuffle test data
)

```

Found 30 files belonging to 6 classes.

```

# Normalize data
normalization_layer = tf.keras.layers.Rescaling(1./255)

# Train data
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))

# Validation data
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))

# Test data
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))

```

```

# Build CNN Model
from tensorflow.keras import layers, models

model = models.Sequential([

    # Conv Layer 1

```

```

layers.Conv2D(32, (3,3), strides=(1,1), padding='same', activation='relu', input_shape=(128,128,3)),
layers.MaxPooling2D(pool_size=(2,2), strides=(2,2)),

# Conv Layer 2
layers.Conv2D(32, (3,3), strides=(1,1), padding='same', activation='relu'),
layers.MaxPooling2D(pool_size=(2,2), strides=(2,2)),

# Flatten
layers.Flatten(),

# Dense Layers
layers.Dense(128, activation='relu'),
layers.Dense(64, activation='relu'),

# Output Layer
layers.Dense(len(classes), activation='softmax')
])

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

# Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

checkpoint = ModelCheckpoint("best_model.h5", save_best_only=True)
early_stop = EarlyStopping(patience=10)

```

```

# Train Model
# Model learns from train_ds
# After each epoch, tested from val_ds

history = model.fit(
    train_ds, # model learns from train_ds
    validation_data=val_ds, # then, tested on validation_ds
    epochs=250,
    batch_size=16,
    callbacks=[checkpoint, early_stop]
)

```

```

Epoch 1/250
3/3 ————— 0s 387ms/step - accuracy: 0.1250 - loss: 2.6632WARNING:absl:You are saving your model as an HDF5 file v
3/3 ————— 4s 702ms/step - accuracy: 0.1250 - loss: 3.2094 - val_accuracy: 0.1667 - val_loss: 2.0852
Epoch 2/250
3/3 ————— 0s 388ms/step - accuracy: 0.2222 - loss: 2.6486WARNING:absl:You are saving your model as an HDF5 file v
3/3 ————— 2s 609ms/step - accuracy: 0.2917 - loss: 2.5904 - val_accuracy: 0.1667 - val_loss: 1.8329
Epoch 3/250
3/3 ————— 0s 379ms/step - accuracy: 0.2957 - loss: 1.8172WARNING:absl:You are saving your model as an HDF5 file v
3/3 ————— 2s 535ms/step - accuracy: 0.2778 - loss: 1.7879 - val_accuracy: 0.1111 - val_loss: 1.7393
Epoch 4/250
3/3 ————— 0s 381ms/step - accuracy: 0.5608 - loss: 1.6032WARNING:absl:You are saving your model as an HDF5 file v
3/3 ————— 2s 538ms/step - accuracy: 0.5417 - loss: 1.5982 - val_accuracy: 0.1667 - val_loss: 1.7236
Epoch 5/250
3/3 ————— 3s 830ms/step - accuracy: 0.4444 - loss: 1.4716 - val_accuracy: 0.1667 - val_loss: 1.7490
Epoch 6/250
3/3 ————— 4s 487ms/step - accuracy: 0.3333 - loss: 1.2919 - val_accuracy: 0.1667 - val_loss: 1.9390
Epoch 7/250
3/3 ————— 2s 552ms/step - accuracy: 0.5278 - loss: 1.0651 - val_accuracy: 0.2778 - val_loss: 1.8687
Epoch 8/250
3/3 ————— 2s 517ms/step - accuracy: 0.7917 - loss: 0.8700 - val_accuracy: 0.2222 - val_loss: 2.1966
Epoch 9/250
3/3 ————— 4s 969ms/step - accuracy: 0.6944 - loss: 0.7699 - val_accuracy: 0.1667 - val_loss: 1.9464
Epoch 10/250
3/3 ————— 8s 2s/step - accuracy: 0.8333 - loss: 0.6126 - val_accuracy: 0.3333 - val_loss: 1.7958
Epoch 11/250
3/3 ————— 3s 721ms/step - accuracy: 0.8889 - loss: 0.4939 - val_accuracy: 0.3333 - val_loss: 2.2160
Epoch 12/250
3/3 ————— 2s 509ms/step - accuracy: 0.9028 - loss: 0.3619 - val_accuracy: 0.1667 - val_loss: 2.5433
Epoch 13/250
3/3 ————— 3s 962ms/step - accuracy: 0.9583 - loss: 0.2197 - val_accuracy: 0.3889 - val_loss: 1.8805
Epoch 14/250
3/3 ————— 4s 1s/step - accuracy: 1.0000 - loss: 0.1283 - val_accuracy: 0.3889 - val_loss: 2.1978

```

```
# # Evaluate model - val_ds
# loss, accuracy = model.evaluate(val_ds)
# print("Accuracy:", accuracy)
```

```
1/1 ----- 0s 191ms/step - accuracy: 0.5556 - loss: 2.1598
Accuracy: 0.555555820465088
```

```
# Evaluate model - test_ds
# REAL PERFORMANCE
loss, accuracy = model.evaluate(test_ds)
print("Model Accuracy (Final Test on 'TEST DATA'):\n", accuracy)
```

```
1/1 ----- 1s 552ms/step - accuracy: 0.6333 - loss: 1.1470
Model Accuracy (Final Test on 'TEST DATA'):\n
0.6333333253860474
```

```
# Predictions on Test Data
y_pred = model.predict(test_ds)
print(y_pred)
y_pred_classes = np.argmax(y_pred, axis=1)
print(y_pred_classes)
print(classes)
```

```
1/1 ----- 0s 446ms/step
[[6.16113961e-01 2.37471983e-01 1.02769963e-01 4.32887971e-02
 3.46005327e-05 3.20731429e-04]
 [4.42267030e-01 4.79523867e-01 2.91658435e-02 4.28140052e-02
 1.46273218e-04 6.08307496e-03]
 [9.49087620e-01 5.98012004e-03 2.84988564e-02 1.63747426e-02
 2.29021362e-06 5.63767680e-05]
 [9.80823934e-01 1.34449336e-03 4.10016440e-03 1.37049612e-02
 5.63847038e-07 2.58511373e-05]
 [9.91301239e-01 3.23682121e-04 4.41006012e-03 3.93670006e-03
 1.36645880e-07 2.81060493e-05]
 [6.32477701e-02 8.06337535e-01 7.73056671e-02 4.72964756e-02
 9.81183024e-04 4.83126799e-03]
 [6.12048805e-01 5.47871105e-02 3.28492880e-01 4.17998806e-03
 2.78828556e-05 4.63342847e-04]
 [9.49189961e-02 2.02729300e-01 6.73644543e-01 2.31545158e-02
 8.58904037e-04 4.69370559e-03]
 [9.22750011e-02 5.03527224e-01 3.82221580e-01 1.86641812e-02
 3.92717920e-04 2.91926553e-03]
 [1.37187988e-01 6.33190334e-01 1.54804662e-01 6.06810264e-02
 6.04982581e-03 8.08607507e-03]
 [2.26402536e-01 1.05240650e-01 6.45812333e-01 2.20511574e-02
 5.85321686e-06 4.87543992e-04]
 [6.91377297e-02 3.13530862e-01 6.14980042e-01 1.69712305e-03
 1.38871532e-04 5.15360036e-04]
 [2.45932043e-02 5.50864153e-02 9.19316888e-01 7.02571764e-04
 6.08710689e-05 2.40104288e-04]
 [4.88791138e-01 8.45346227e-02 4.19510305e-01 7.08175404e-03
 7.74687464e-07 8.14135710e-05]
 [3.55442494e-01 2.16538146e-01 3.79210263e-01 4.56899554e-02
 5.41583635e-04 2.57749716e-03]
 [3.16166756e-04 9.01204124e-02 9.52389673e-05 8.38240445e-01
 5.63560314e-02 1.48716765e-02]
 [2.74573034e-03 5.17211497e-01 4.13568178e-03 4.43144500e-01
 1.77903045e-02 1.49722910e-02]
 [1.84118352e-03 8.77170824e-03 2.31906248e-04 7.89715409e-01
 1.73624873e-01 2.58147959e-02]
 [1.04749796e-03 7.01078260e-03 1.82373369e-05 9.72708642e-01
 1.62774418e-02 2.93728057e-03]
 [8.50770622e-02 1.61891058e-01 3.41002480e-03 7.35182941e-01
 7.86682311e-03 6.57210546e-03]
 [1.74872014e-06 9.35934950e-05 7.91612942e-10 7.01536238e-02
 7.03782916e-01 2.25968063e-01]
 [9.08367801e-03 3.40234667e-01 2.11887746e-05 4.47674513e-01
 1.48688138e-01 5.42978682e-02]
 [1.64483950e-04 8.86307843e-03 5.04339198e-07 6.23323143e-01
 3.40431899e-01 2.72169523e-02]
 [1.08943124e-04 1.15222810e-02 1.01762872e-07 1.46515518e-01
 6.73752129e-01 1.68101013e-01]
 [1.47982882e-05 7.04042090e-04 5.05470732e-09 1.28691226e-01
 5.81835508e-01 2.88754493e-01]
 [1.93151078e-04 7.26500293e-03 4.55575957e-07 2.25986373e-02
 4.44700032e-01 5.25242686e-01]
 [1.19814858e-01 4.51869279e-01 3.64466459e-01 5.52831925e-02
 4.04998520e-03 4.51636594e-03]
 [7.72044539e-01 9.87379402e-02 3.93991061e-02 8.20655823e-02
```

```
5.32931706e-04 7.21989991e-03]
```

```
# Get True Labels
y_true = np.concatenate([y for x, y in test_ds], axis=0)
print(y_true)
```

```
[0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4 5 5 5 5 5]
```

```
# Classification Report
from sklearn.metrics import classification_report

print(classification_report(y_true, y_pred_classes))
```

	precision	recall	f1-score	support
0	0.57	0.80	0.67	5
1	0.43	0.60	0.50	5
2	0.80	0.80	0.80	5
3	0.67	0.80	0.73	5
4	0.75	0.60	0.67	5
5	1.00	0.20	0.33	5
accuracy			0.63	30
macro avg	0.70	0.63	0.62	30
weighted avg	0.70	0.63	0.62	30

--- Overall Performance ---

- Accuracy: 0.63 (63%) -> Out of 30 test images, about 19 were predicted correctly.
- Precision: Out of predicted, how many were correct.
- Recall: Out of actual, how many were found.
- F1-Score: Balance of precision & recall.
- Support: Number of actual samples.

- Moderately Good (63%)
- Strong on some classes
- Weak on few -> needs tuning

"The model has achieved an accuracy of 63% on the test dataset. Certain classes such as class 2 and class 3 show strong performance with high precision and recall. However, class 1 and class 5 show weaker performance, indicating that the model struggles to generalize across all categories."

```
model.save("cnn_model.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format

```
from tensorflow.keras.models import load_model
```

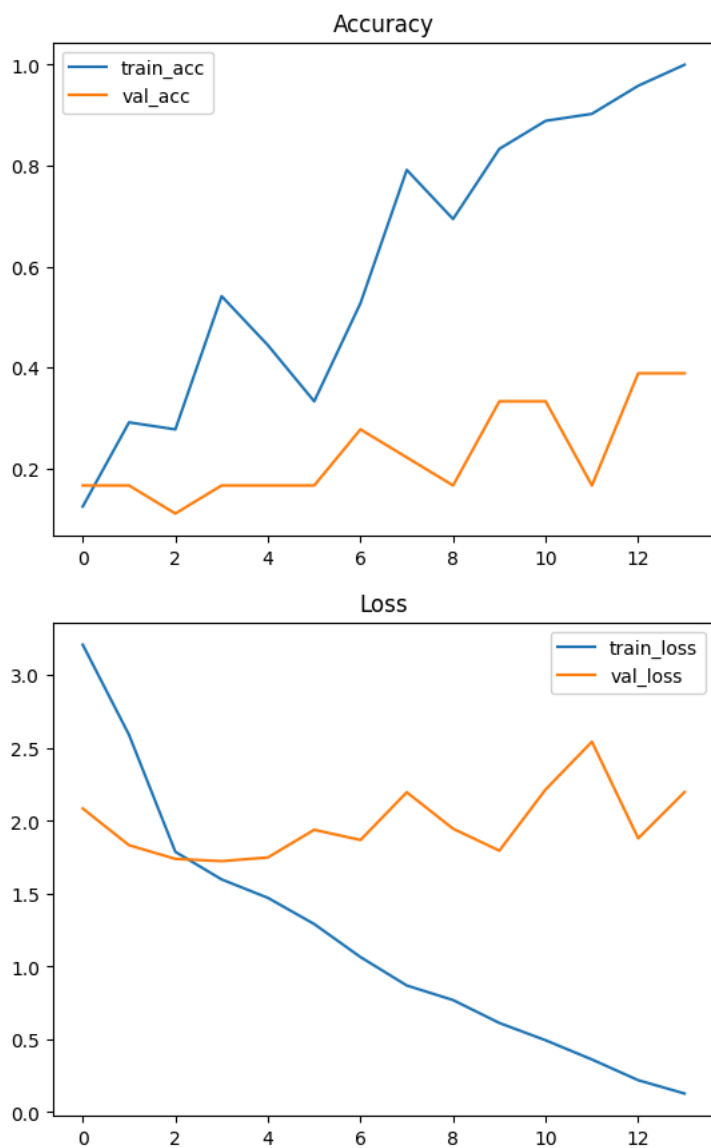
```
model = load_model("cnn_model.h5")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until

```
# Data Visualization
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.legend()
plt.title("Accuracy")
plt.show()
```

```
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.legend()
plt.title("Loss")
plt.show()
```



--- Insights ---

"Fig 1"

Blue Line -> Training Accuracy

Orange Line -> Validation Accuracy

Training Accuracy (starts low - gradually increases to 1.0) -> model is learning the training data very well.

Validation Accuracy (stays low (0.1 - 0.4)) -> model is not performing well on unseen data.

Train = 100% | Val = 40% (Classic case of Overfitting)

"Fig 2"

Blue Line -> Training Loss

Orange Line -> Validation Loss

Training Loss (decreases steadily (almost to 0)) -> model is fitting training data perfectly.

Validation Loss (fluctuates between 1.7 - 2.5) -> model is struggling on new data.

Training Loss (low) but validation loss (high) (Overfitting)

"The training accuracy increases significantly while the validation accuracy remains low and fluctuates. Similarly, training loss decreases steadily whereas validation loss does not improve. This indicates that the model is overfitting, meaning it performs well on training data but poorly on unseen data."

```

from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
import numpy as np

# Convert report to dictionary
report = classification_report(y_true, y_pred_classes, output_dict=True)

# Extract class labels
classes = [k for k in report.keys() if k.isdigit()]

precision = [report[c]['precision'] for c in classes]
recall = [report[c]['recall'] for c in classes]
f1 = [report[c]['f1-score'] for c in classes]

x = np.arange(len(classes))

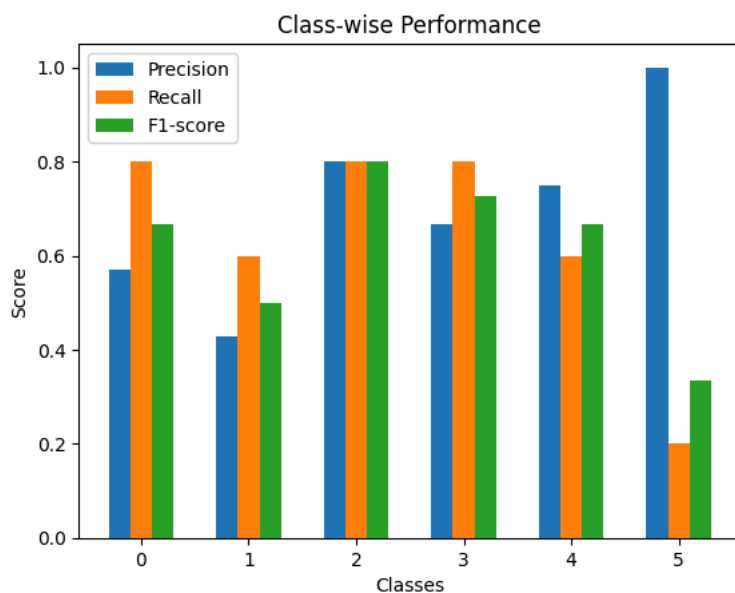
plt.figure()

plt.bar(x - 0.2, precision, width=0.2)
plt.bar(x, recall, width=0.2)
plt.bar(x + 0.2, f1, width=0.2)

plt.xticks(x, classes)
plt.xlabel("Classes")
plt.ylabel("Score")
plt.title("Class-wise Performance")
plt.legend(["Precision", "Recall", "F1-score"])

plt.show()

```



```

from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

